# Django Assignment

Student: Junayed | Narayanganj Polytechnic Institute

## Question 1: Django Template Engine, Tags and Filters

### What is the Django Template Engine?

The Django Template Engine (DTL) is the part of Django that creates the HTML page the user sees. A template is a normal HTML file with some special code inside it. The engine takes the template, puts the real data from the View inside it, and gives back the final HTML page.

It helps us keep Python code (logic) and HTML (design) in separate places. Templates are kept inside a *templates* folder. We use the **render()** function in the View to send data to a template.

```
# views.py
from django.shortcuts import render

def home(request):
    context = {"name": "Junayed", "marks": 85}
    return render(request, "home.html", context)
```

```
<!-- templates/home.html -->
<h1>Hello, {{ name }}</h1>
<p>Your marks: {{ marks }}</p>
```

Here **{{ name }}** is a variable. Django replaces it with the real value, so the browser shows "Hello, Junayed".

### Template Tags

Template tags are written inside **{% %}**. They do work like logic: loops, conditions, including other files, making links, and so on. Some tags need a closing tag (for example *{% endif %}*).

```
{% if marks >= 80 %}
    <p>Grade: A+</p>
{% else %}
    <p>Keep trying!</p>
{% endif %}

{% for subject in subjects %}
    <li>{{ subject }}</li>
{% endfor %}

<a href="{% url 'home' %}">Go to Home</a>
{% extends "base.html" %}
{% block content %} ... {% endblock %}
```

### Template Filters

Template filters are written with a pipe symbol **|** after a variable. They change how a value looks. They do not do logic, they only change the value. Some filters take an argument after a colon.

```
{{ name|upper }}                -> JUNAYED
{{ name|lower }}                -> junayed
{{ title|title }}               -> Django Is Fun
{{ text|truncatewords:5 }}      -> first 5 words...
{{ today|date:"d M Y" }}        -> 08 Nov 2026
```

```
{{ items|length }}              -> number of items
{{ value|default:"No data" }}   -> shows text if value is empty
```

### Difference between Tags and Filters

| Point | Template Tag | Template Filter |
|---|---|---|
| Syntax | {% tag_name %} | {{ value\|filter_name }} |
| Purpose | Controls logic and structure | Changes the look of a value |
| Examples | if, for, url, extends, block, include | upper, lower, date, length, default |
| Closing tag | Some need one, like {% endif %} | Never needed |
| Input | Can work without any variable | Always works on a value |

## Question 2: Django Middleware

### What is Middleware?

Middleware is a small layer that sits between the **request** and the **response**. Every request first goes through all middleware, then reaches the View. When the View sends the answer, the response goes back through the middleware again. We can think of it like a security gate: everyone must pass it, both while going in and coming out.

Middleware is used for things like security, login checking, sessions, logging and blocking bad requests. It is listed in the **MIDDLEWARE** list in *settings.py*.

### Built-in Middleware

These are already made by Django, and we only need to add them in settings. Examples:

- **SecurityMiddleware:** adds security headers and HTTPS settings.
- **SessionMiddleware:** turns on sessions, so Django can remember the user.
- **CommonMiddleware:** handles things like adding a slash at the end of URLs.
- **CsrfViewMiddleware:** protects forms from CSRF attacks.
- **AuthenticationMiddleware:** adds *request.user* to every request.
- **MessageMiddleware:** allows one-time messages like "Saved successfully".

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

### Custom Middleware

A custom middleware is one that we write ourselves for our own need, for example to print how long a request took, or to block some IP addresses. We write it as a class (or function) and then add its path to the MIDDLEWARE list.

```
# myapp/middleware.py
import time

class TimerMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response      # runs once at server start

    def __call__(self, request):
        start = time.time()                   # before the view runs
        response = self.get_response(request) # goes to next layer / view
        end = time.time()                     # after the view runs
        print(f"Time taken: {end - start:.3f} seconds")
        return response

# settings.py
MIDDLEWARE = [
    ...
    "myapp.middleware.TimerMiddleware",
]
```

### Built-in vs Custom Middleware

| Point | Built-in Middleware | Custom Middleware |
|---|---|---|
| Made by | Django team | The developer (us) |
| Where it is | Inside Django package | Inside our own project/app |
| Purpose | Common jobs: security, session, auth, CSRF | Special jobs for our project |
| Setup | Only add the name in settings | Write the class, then add it in settings |
| Example | SessionMiddleware | TimerMiddleware |

### Middleware Execution Order

Middleware runs in the same order as the MIDDLEWARE list **from top to bottom** when the request comes in. Then the View runs. After that, the response goes back **from bottom to top** (reverse order).

```
Request  --> Middleware 1 --> Middleware 2 --> Middleware 3 --> VIEW
Response <-- Middleware 1 <-- Middleware 2 <-- Middleware 3 <-- VIEW
```

**Important:** The order matters. For example, SessionMiddleware must be above AuthenticationMiddleware, because authentication needs the session to know who the user is. If any middleware returns a response early, the request stops there and does not go deeper.

## Question 3: Django ModelForm and GET / POST

### What is a ModelForm?

A ModelForm is a special form that is made directly from a Model. Django reads the model fields and creates the form fields by itself. So we do not need to write every field again. It also has a **save()** method that saves the data to the database in one line.

## How to create a ModelForm

**Step 1:** Make a model.

```
# models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    age = models.IntegerField()

    def __str__(self):
        return self.name
```

**Step 2:** Make a form file and create the ModelForm.

```
# forms.py
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ["name", "email", "age"]     # or "__all__"
```

**Step 3:** Use the form in a View and handle GET and POST.

```
# views.py
from django.shortcuts import render, redirect
from .forms import StudentForm

def add_student(request):
    if request.method == "POST":
        form = StudentForm(request.POST)      # data sent by user
        if form.is_valid():
            form.save()                       # save in database
            return redirect("student_list")
    else:
        form = StudentForm()                  # empty form for GET
    return render(request, "add_student.html", {"form": form})
```

**Step 4:** Show the form in the template. **{% csrf_token %}** is a must for POST forms.

```
<!-- add_student.html -->
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>
```

## GET and POST requests

| Point | GET | POST |
|---|---|---|
| Use | To get or show a page / search data | To send data to the server (create / update) |
| Data location | Shown in the URL (?name=ali) | Hidden in the request body |

| | | |
|---|---|---|
| Safety | Less safe, do not use for passwords | More safe, needs CSRF token |
| Read in Django | request.GET | request.POST |
| Data size | Small | Can be large |

**How it works:** When the user first opens the page, the browser sends a **GET** request and the View shows an empty form. When the user fills the form and clicks Save, the browser sends a **POST** request. The View checks the data with *is_valid()*. If it is correct, the data is saved. If not, the form is shown again with error messages.

```
# GET example: search form  (URL: /search/?q=django)
def search(request):
    query = request.GET.get("q")
    return render(request, "search.html", {"query": query})
```

## Question 4: Static Files vs Media Files

### Difference

**Static files** are the files that are part of our project and do not change, like CSS, JavaScript and our logo. The developer adds them. **Media files** are the files that users upload while the site is running, like profile pictures or documents. They can change anytime.

| Point | Static Files | Media Files |
|---|---|---|
| Meaning | Files of the project itself | Files uploaded by users |
| Examples | CSS, JS, logo, icons | Profile photo, PDF, video |
| Changes? | No, only when developer changes | Yes, any time by users |
| Settings | STATIC_URL, STATICFILES_DIRS, STATIC_ROOT | MEDIA_URL, MEDIA_ROOT |
| Template tag | {% static 'file' %} | {{ obj.image.url }} |

### Configure image upload step by step

**Step 1:** Install Pillow, because ImageField needs it.

```
pip install pillow
```

**Step 2:** Add settings in *settings.py*.

```
import os

MEDIA_URL = "/media/"                          # URL to open uploaded files
MEDIA_ROOT = os.path.join(BASE_DIR, "media")    # folder where files are saved

STATIC_URL = "/static/"
```

**MEDIA_ROOT** is the real folder on the computer where uploaded files are saved. **MEDIA_URL** is the web address used to open those files in the browser.

**Step 3:** Add an ImageField in the model.

```
# models.py
```

```
class Profile(models.Model):
    name = models.CharField(max_length=100)
    photo = models.ImageField(upload_to="photos/")   # saved in media/photos/
```

**Step 4:** Make a ModelForm and handle the upload in the View. Use **request.FILES** for files.

```
# forms.py
class ProfileForm(forms.ModelForm):
    class Meta:
        model = Profile
        fields = ["name", "photo"]

# views.py
def upload(request):
    if request.method == "POST":
        form = ProfileForm(request.POST, request.FILES)   # FILES is important
        if form.is_valid():
            form.save()
            return redirect("profile_list")
    else:
        form = ProfileForm()
    return render(request, "upload.html", {"form": form})
```

**Step 5:** In the HTML form, add **enctype="multipart/form-data"**. Without it, the file will not be sent.

```
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>

<!-- show the image -->
<img src="{{ profile.photo.url }}" width="200">
```

**Step 6:** Add the media URL in the main *urls.py* (for development only).

```
# project urls.py
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    ...
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Now an uploaded image is saved at *media/photos/image.jpg* and can be opened at *http://127.0.0.1:8000/media/photos/image.jpg*.

# Question 5: Authentication and Session Middleware

## Django Authentication

Authentication means checking **who the user is**. Django has a ready-made system for this in *django.contrib.auth*. It gives us the User model, password hashing (passwords are never saved as plain text), login, logout, permissions and groups.

**AuthenticationMiddleware** adds the **request.user** object to every request. So in any View or template we can know who the current user is. If nobody has logged in, it is an AnonymousUser.

## Session Middleware

HTTP does not remember anything between two requests. **SessionMiddleware** solves this. It gives each visitor a unique **session id** and saves it in the browser as a cookie. The real data (like which user is logged in) is kept on the server, in the database table *django_session* by default. On the next request, Django reads the cookie, finds the session on the server, and knows the user again.

## How Login works

The user sends username and password with a POST form. Django checks them with **authenticate()**. If they are correct, **login()** creates a session and saves the user id in it. The browser gets a session cookie.

```
# views.py
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect

def user_login(request):
    if request.method == "POST":
        username = request.POST["username"]
        password = request.POST["password"]
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)          # creates the session
            return redirect("dashboard")
        else:
            return render(request, "login.html", {"error": "Wrong username or password"})
    return render(request, "login.html")
```

## How Logout works

**logout()** deletes the session data of the user from the server and clears the cookie. After this, the user is an anonymous user again.

```
def user_logout(request):
    logout(request)                       # deletes the session
    return redirect("login")
```

## Protecting pages

The **@login_required** decorator lets only logged-in users open a page. Others are sent to the login page.

```
@login_required(login_url="login")
def dashboard(request):
    return render(request, "dashboard.html", {"user": request.user})

# settings.py
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "dashboard"

<!-- in template -->
{% if user.is_authenticated %}
```

```
    <p>Welcome, {{ user.username }}</p>
    <a href="{% url 'logout' %}">Logout</a>
{% else %}
    <a href="{% url 'login' %}">Login</a>
{% endif %}
```

## Session Management

We can also store our own small data in the session using **request.session**. It works like a Python dictionary.

```
def set_theme(request):
    request.session["theme"] = "dark"          # save data
    return redirect("home")

def get_theme(request):
    theme = request.session.get("theme", "light")   # read data
    return render(request, "home.html", {"theme": theme})

del request.session["theme"]                   # delete one item
request.session.flush()                        # delete whole session

# settings.py
SESSION_COOKIE_AGE = 1209600                   # 2 weeks in seconds
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
```

## Complete flow

```
1. User submits login form (POST)
2. authenticate() checks username and password
3. login() creates a session and saves user id
4. Browser stores the session id cookie
5. Next requests: SessionMiddleware reads the cookie
6. AuthenticationMiddleware sets request.user
7. logout() deletes the session -> user is logged out
```

*Thank you for reading!*